Asynchronous JavaScript

Complete Notes — Fundamentals to Advanced

Callbacks | Promises | async / await | Event Loop | Error Handling | Interview Q&A

Topics covered in this guide

Table of Contents

1. What is Asynchronous JavaScript?

The Problem: JavaScript is Single-Threaded

JavaScript runs on a single thread — it can only do **one thing at a time**. This means if you run a slow operation (like fetching data from a server), the entire program freezes until that operation finishes. No clicks, no animations, nothing.

✔ KEY POINT
Synchronous = "do this, wait, then do next"
Asynchronous = "start this, move on, come back when it's done"

Synchronous Code Problem — Visual

```
// SYNC — BAD for slow operations
const data = fetchFromServer();  // freezes here for 3 seconds!
console.log('This prints after 3s');
console.log('User cannot click anything during that time');

// ASYNC — GOOD
fetchFromServer().then(data => {
  console.log('Got data:', data);  // runs when ready
});
console.log('This prints immediately!');  // doesn't wait
```

What Counts as 'Async' in JavaScript?

Category	Examples	Why Async?
Network requests	fetch(), XMLHttpRequest	Server response takes time
Timers	setTimeout, setInterval	Delay is intentional
File I/O (Node.js)	fs.readFile()	Disk is slow
User events	click, keypress	Unpredictable timing
Animations	requestAnimationFrame	Tied to screen refresh
Databases	MongoDB, IndexedDB queries	Storage is slow

The 3 Ways to Handle Async in JS

JavaScript evolved through three approaches — each solving the problems of the previous one:

Approach	Era	Solves	Problem
Callbacks	Pre-2015	Async basics	Callback hell, hard to read
Promises	ES2015 (ES6)	Callback hell	Verbose chaining

async/await	ES2017 (ES8)	Readability	Still promises under the hood

2. JavaScript Runtime Environment

To understand async JS, you must understand what runs it. The JS runtime is NOT just the engine — it's an ecosystem of parts working together.

The 4 Key Components

Call Stack	Where JS executes code. Functions pushed when called, popped when returned. LIFO (Last In, First Out). Only one stack — single thread.
Web APIs / Node APIs	Browser (or Node) provides these: setTimeout, fetch, DOM events, fs.readFile. They handle async work OUTSIDE the JS engine. JS hands off the task and continues.
Callback / Task Queue (Macro)	When a Web API finishes (e.g. timer fires, fetch completes), it puts the callback here. Processed one at a time after the call stack is empty. Also called the 'Message Queue'.
Microtask Queue	Higher priority than the task queue. Holds Promise .then()/.catch() callbacks and queueMicrotask(). FULLY drained before the next macro-task runs.

How They Work Together — Step by Step

```
console.log('1');               // → Call Stack

setTimeout(() => {
  console.log('2');             // → Web API (timer) → Task Queue → Stack
}, 0);

Promise.resolve().then(() => {
  console.log('3');             // → Microtask Queue → Stack
});

console.log('4');               // → Call Stack

// Output: 1, 4, 3, 2
// Why? Stack runs 1 & 4 first. Then microtasks (3). Then macro-tasks (2).
```

NOTE

Even setTimeout(fn, 0) is NOT immediate. It goes to the Task Queue and only runs after the call stack is empty AND all microtasks are done.

3. The Event Loop

The Event Loop is the mechanism that coordinates all 4 components. It constantly checks: *"Is the call stack empty? If yes, grab the next task."*

The Event Loop Algorithm (simplified)

Step 1	Execute all synchronous code (fill and empty the Call Stack)
Step 2	Check the Microtask Queue — run ALL microtasks until it's empty
Step 3	Check the Task Queue (Macro) — run ONE task
Step 4	Go back to Step 2 (repeat forever)

Macro-tasks vs Micro-tasks

Type	Examples	Priority	Drained how?
Microtask	Promise.then/catch, queueMicrotask, MutationObserver	HIGH — runs first	ALL at once before next macro
Macro-task	setTimeout, setInterval, setImmediate, I/O callbacks, UI rendering	LOW — runs after micro	ONE per event loop turn

Classic Tricky Interview Example

```
console.log('start');

setTimeout(() => console.log('timeout'), 0);

Promise.resolve()
  .then(() => console.log('promise 1'))
  .then(() => console.log('promise 2'));

console.log('end');

// Output order:
// start  → sync
// end    → sync
// promise 1 → microtask
// promise 2 → microtask (queued by previous .then)
// timeout  → macro-task (last!)
```

■ INTERVIEW

Rule: Microtasks ALWAYS run before macro-tasks.

Promise chains queue new microtasks, so they all complete before any setTimeout fires.

4. Callbacks

What is a Callback?

A **callback** is simply a function passed as an argument to another function, to be called later when an async operation completes.

```
// Basic callback pattern
function fetchUser(id, callback) {
  setTimeout(() => {                    // simulate network delay
    const user = { id, name: 'Alice' };
    callback(null, user);               // Node.js convention: (error, result)
  }, 1000);
}

fetchUser(1, (err, user) => {
  if (err) return console.error(err);
  console.log(user.name);               // Alice
});
```

Callback Hell (Pyramid of Doom)

When you need results from multiple async operations in sequence, callbacks nest deeply:

```
getUser(userId, (err, user) => {
  if (err) handle(err);
  getPosts(user.id, (err, posts) => {
    if (err) handle(err);
    getComments(posts[0].id, (err, comments) => {
      if (err) handle(err);
      getLikes(comments[0].id, (err, likes) => {
        if (err) handle(err);
        console.log(likes);  // finally!
      });
    });
  });
});
// Hard to read, hard to maintain, error handling repeated everywhere
```

■ NOTE

Problems with Callbacks:

1. Deeply nested — hard to read ('pyramid of doom')
2. Error handling must be repeated at every level
3. Can't use try/catch for async errors
4. Hard to run operations in parallel cleanly
5. 'Inversion of control' — you trust the caller to call your callback correctly

Pros & Cons of Callbacks

Pros	Cons
Simple concept — just a function	Callback hell with deep nesting
Works everywhere (no transpilation)	Error handling is repetitive
Low overhead — no Promise wrapper	Inversion of control (trust issue)
Fine for single async operations	Hard to compose or parallelize

5. Promises

What is a Promise?

A **Promise** is an object representing the eventual completion (or failure) of an async operation. It's a placeholder for a future value.

The 3 States of a Promise

Pending	Initial state. Operation has not completed yet.
Fulfilled	Operation succeeded. Has a resolved value.
Rejected	Operation failed. Has a rejection reason (error).

Once settled (fulfilled or rejected), a Promise CANNOT change state.

Creating a Promise

```
const promise = new Promise((resolve, reject) => {
  // do async work...
  const success = true;
  if (success) {
    resolve('Data loaded!');    // → fulfills the promise
  } else {
    reject(new Error('Oops!')); // → rejects the promise
  }
});

// Consuming the promise
promise
  .then(value => console.log(value))   // 'Data loaded!'
  .catch(err => console.error(err))    // handles rejection
  .finally(() => console.log('done')); // always runs
```

Promise Chaining

.then() always returns a NEW Promise, enabling chaining:

```
fetch('/api/user/1')
  .then(response => response.json())      // parse JSON → new Promise
  .then(user => fetch(`/api/posts/${user.id}`)) // fetch posts → new Promise
  .then(response => response.json())
  .then(posts => console.log(posts))
  .catch(err => console.error('Failed:', err))  // catches ANY error above
  .finally(() => hideLoader());
```

✔ KEY POINT
Key: .catch() at the end catches errors from ANY step in the chain.
You don't need error handling at every level (unlike callbacks).

Full Promise API Reference

`Promise.resolve(value)`	Creates an already-fulfilled promise. Useful for wrapping sync values. Promise.resolve(42).then(v => console.log(v)); // 42
`Promise.reject(reason)`	Creates an already-rejected promise. Promise.reject(new Error('fail')).catch(e => console.error(e));
`Promise.all(iterable)`	Waits for ALL promises to fulfill. Rejects fast if ANY rejects. Returns array of all results in input order.
`Promise.allSettled(iterable)`	Waits for ALL to settle (fulfill OR reject). Never rejects. Returns array of {status, value/reason} objects.
`Promise.race(iterable)`	Resolves/rejects as soon as the FIRST one settles. Useful for timeouts.
`Promise.any(iterable)`	Fulfills with FIRST fulfilled. Rejects only if ALL reject (AggregateError). Opposite of Promise.all in rejection behaviour.

6. async / await

What & Why?

async/await is syntactic sugar over Promises. It lets you write async code that LOOKS synchronous, making it far easier to read and reason about.

Syntax Rules

- **async function**: Add 'async' before a function declaration. It now returns a Promise automatically.
- **await**: Pause execution INSIDE an async function until a Promise settles. Can only be used INSIDE async functions.
- **Return value**: Whatever you return from async fn is wrapped in a resolved Promise.
- **Throw**: Throwing inside async fn creates a rejected Promise.

Before vs After

```
// ■■ With Promises ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
function loadUserData(id) {
  return fetch(`/api/users/${id}`)
    .then(res => res.json())
    .then(user => fetch(`/api/posts/${user.id}`))
    .then(res => res.json())
    .catch(err => console.error(err));
}

// ■■ With async/await ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async function loadUserData(id) {
  try {
    const res  = await fetch(`/api/users/${id}`);
    const user = await res.json();
    const postsRes = await fetch(`/api/posts/${user.id}`);
    const posts    = await postsRes.json();
    return posts;
  } catch (err) {
    console.error(err);  // one catch for all errors
  }
}
```

Under the Hood — async/await IS Promises

```
async function example() {
  const result = await somePromise;
  console.log(result);
}

// IS EXACTLY EQUIVALENT TO:
function example() {
  return somePromise.then(result => {
    console.log(result);
  });
}
```

■ NOTE

async/await does NOT make JS multi-threaded. The await keyword pauses only the CURRENT async function, not the entire program. Other code still runs!

Error Handling with async/await

```
// Pattern 1: try/catch (recommended)
async function getData() {
  try {
    const data = await fetch('/api/data');
    return await data.json();
  } catch (err) {
    console.error('Error:', err.message);
    throw err;  // re-throw if caller needs to handle it
  }
}

// Pattern 2: .catch() on the call
const result = await getData().catch(err => null);  // null on error

// Pattern 3: Helper to avoid try/catch everywhere
const [err, data] = await to(getData());  // Go-style error handling
function to(promise) {
  return promise.then(data => [null, data]).catch(err => [err, null]);
}
```

Common Mistake: Forgetting await

```
// BUG — forgot await on fetch
async function bad() {
  const res = fetch('/api');   // res is a PROMISE, not the response!
  const data = res.json();    // Error: res.json is not a function
}

// FIX
async function good() {
  const res  = await fetch('/api');   // actual Response object
  const data = await res.json();      // actual data
}
```

7. Parallel & Concurrent Patterns

The Biggest async/await Mistake: Unnecessary Sequencing

```
// SLOW — sequential (waits 3s total: 1s + 2s)
async function slow() {
  const user  = await getUser();   // 1s
  const posts = await getPosts();  // 2s — waits for getUser unnecessarily!
  return { user, posts };
}

// FAST — parallel with Promise.all (waits only 2s — the longest)
async function fast() {
  const [user, posts] = await Promise.all([getUser(), getPosts()]);
  return { user, posts };
}
```

✔ KEY POINT

Only await sequentially when one result DEPENDS on the previous one.
If they're independent, run them in parallel with Promise.all.

Promise.all — All or Nothing

```
const results = await Promise.all([
  fetch('/api/users'),
  fetch('/api/posts'),
  fetch('/api/comments'),
]);
// If ANY fails → entire Promise.all rejects!
// Use when all results are REQUIRED
```

Promise.allSettled — Wait for All, No Fail

```
const results = await Promise.allSettled([
  fetch('/api/users'),
  fetch('/api/posts'),   // this one might fail
]);

results.forEach(result => {
  if (result.status === 'fulfilled') console.log(result.value);
  else console.error(result.reason);
});
// Use when you want ALL results even if some fail
```

Promise.race — First One Wins

```
// Timeout pattern
function timeout(ms) {
  return new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout!')), ms)
  );
}

const result = await Promise.race([
  fetch('/api/slow-endpoint'),
  timeout(5000)             // reject if fetch takes > 5s
]);
```

Promise.any — First SUCCESS Wins

```
// Try multiple mirrors, use whichever responds first
const data = await Promise.any([
  fetch('https://mirror1.com/data'),
  fetch('https://mirror2.com/data'),
  fetch('https://mirror3.com/data'),
]);
// Rejects only if ALL three fail (AggregateError)
```

Quick Comparison Table

Method	Fulfills when	Rejects when	Best for
Promise.all	ALL fulfill	ANY rejects (fast-fail)	Required parallel data
Promise.allSettled	ALL settle (any state)	Never rejects	Optional parallel data
Promise.race	FIRST settles (any state)	FIRST rejects	Timeouts, fastest wins
Promise.any	FIRST fulfills	ALL reject	Fallbacks, redundancy

8. Real-World Patterns

Retry with Exponential Backoff

```
async function fetchWithRetry(url, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return await res.json();
    } catch (err) {
      if (i === maxRetries - 1) throw err;         // last attempt
      const delay = 1000 * Math.pow(2, i);         // 1s, 2s, 4s
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

Debounce (async-friendly)

Delay execution until the user stops typing/clicking:

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

const search = debounce(async (query) => {
  const results = await fetch(`/api/search?q=${query}`);
  displayResults(await results.json());
}, 300);

input.addEventListener('input', e => search(e.target.value));
```

Throttle

Execute at most once per time window:

```
function throttle(fn, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}

window.addEventListener('scroll', throttle(updateUI, 100));
```

AbortController — Cancel Fetch Requests

```
const controller = new AbortController();
const { signal } = controller;

// Start fetch with a signal
fetch('/api/long-operation', { signal })
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => {
    if (err.name === 'AbortError') console.log('Cancelled!');
    else throw err;
  });

// Cancel after 5 seconds
setTimeout(() => controller.abort(), 5000);

// React pattern: cancel on unmount
useEffect(() => {
  const controller = new AbortController();
  fetchData({ signal: controller.signal });
  return () => controller.abort();   // cleanup
}, []);
```

Async Iteration (for await...of)

```
// Reading a stream or paginated API
async function* paginate(baseUrl) {
  let page = 1;
  while (true) {
    const res  = await fetch(`${baseUrl}?page=${page}`);
    const data = await res.json();
    if (!data.items.length) return;
    yield data.items;
    page++;
  }
}

for await (const items of paginate('/api/products')) {
  processItems(items);
}
```

9. Common Mistakes & Gotchas

1. Unhandled Promise Rejections

```
const p = fetch('/api');  // rejection never caught!
// Always handle errors:
const p = fetch('/api').catch(console.error);
```

2. await in forEach (doesn't work!)

```
// WRONG — forEach ignores returned promises
items.forEach(async item => { await process(item); });

// RIGHT — use for...of or Promise.all
for (const item of items) { await process(item); }
// OR parallel:
await Promise.all(items.map(item => process(item)));
```

3. await outside async function

```
// WRONG — top-level await not allowed (unless ESM module)
const data = await fetchData();  // SyntaxError

// RIGHT — wrap in async IIFE or use top-level await in .mjs
(async () => { const data = await fetchData(); })();
```

4. Not checking response.ok in fetch()

```
// fetch() only rejects on NETWORK errors, not 404/500!
const res = await fetch('/api');
// res.ok is false for 4xx/5xx but NO error thrown

// RIGHT:
const res = await fetch('/api');
if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
```

5. Creating Promise inside Promise (anti-pattern)

```
// WRONG — explicit promise construction anti-pattern
function delay(ms) {
  return new Promise(resolve => {
    return new Promise(r => setTimeout(r, ms))  // unnecessary!
      .then(resolve);
  });
}
// RIGHT:
const delay = ms => new Promise(r => setTimeout(r, ms));
```

10. Interview Q&A — Rapid Fire

Q: What is the difference between synchronous and asynchronous code?

Sync code blocks execution until it completes. Async code initiates an operation and moves on, handling the result later via callbacks, promises, or async/await.

Q: What is the Event Loop?

The mechanism that coordinates the Call Stack, Web APIs, Microtask Queue, and Task Queue. It checks if the stack is empty, runs all microtasks, then picks one macro-task — repeating forever.

Q: What is the difference between the Microtask Queue and the Task Queue?

Microtasks (Promise callbacks) have higher priority and run FULLY after each task before the next macro-task. Task Queue holds setTimeout/setInterval callbacks and runs one per event loop turn.

Q: What is the output of: setTimeout(fn,0) vs Promise.resolve().then(fn)?

Promise.then(fn) runs FIRST. Microtasks (Promises) always execute before macro-tasks (setTimeout), even with a 0ms delay.

Q: What does async keyword do to a function?

It makes the function always return a Promise. Any returned value is wrapped in a resolved Promise. Any thrown error becomes a rejected Promise.

Q: Can you use await without async?

Only at the top level of ES Modules (top-level await). Otherwise, await must be inside an async function — it's a SyntaxError otherwise.

Q: What is the difference between Promise.all and Promise.allSettled?

Promise.all rejects immediately if ANY promise rejects (fast-fail). Promise.allSettled waits for ALL to complete regardless of success or failure, returning an array of {status, value/reason}.

Q: What happens if you don't await a Promise?

The promise runs in the background (fire-and-forget). Errors won't be caught. Execution continues immediately with the Promise object, not the resolved value.

Q: What is callback hell and how do Promises solve it?

Callback hell is deeply nested callbacks making code unreadable. Promises solve it by enabling flat chaining (.then().then()) and centralized error handling with .catch().

Q: Why doesn't async/await work with forEach?

forEach ignores returned promises from its callback. The loop finishes before any awaited operations complete. Use for...of for sequential, or Promise.all(arr.map()) for parallel execution.

Q: How do you handle errors in async/await?

Use try/catch around await calls. Or attach .catch() to the async function call. Or use a helper like to(promise) that returns [err, data].

Q: What is AbortController?

A browser API to cancel in-flight fetch requests. Create one, pass signal to fetch(), call abort() to cancel. The fetch rejects with an AbortError.

Q: What is the difference between Promise.race and Promise.any?

race settles as soon as the FIRST promise settles (whether fulfilled or rejected). any fulfills with the FIRST fulfilled promise, only rejecting if ALL reject.

Q: Can async operations run truly in parallel in JavaScript?

Not in the JS engine (single-threaded). But Web APIs (fetch, timers) run outside the engine, so multiple fetches can be in-flight simultaneously. Promise.all leverages this.

Q: What is a microtask?

A task in the Microtask Queue. Includes Promise .then/.catch callbacks and queueMicrotask(). Runs after the current script and before the next macro-task.

Quick Reference Cheat Sheet

Concept	One-liner
`Callback`	Function passed to another fn, called when async op completes
`Promise`	Object representing a future value. Pending → Fulfilled | Rejected
`.then()`	Runs on fulfillment. Returns new Promise. Chainable.
`.catch()`	Runs on rejection. Catches errors from entire chain above it.
`.finally()`	Runs always. Doesn't receive value. For cleanup.
`async fn`	Always returns Promise. Lets you use await inside.
`await`	Pauses async fn until Promise settles. Only inside async fn.
`Promise.all`	All succeed or fast-fail. Use for parallel required data.
`Promise.allSettled`	All settle, never fails. Use for parallel optional data.
`Promise.race`	First to settle wins (success or failure). Use for timeouts.
`Promise.any`	First to SUCCEED wins. All must fail to reject.
`Event Loop`	Stack empty → drain microtasks → run one macro-task → repeat
`Microtask queue`	Promise callbacks. Runs before next macro-task.
`Task queue`	setTimeout, I/O callbacks. Runs one per event loop turn.
`AbortController`	Cancel fetch requests. Pass signal, call abort().

✔ KEY POINT

Final Tips:

- Always handle Promise rejections (unhandled rejection = silent bug)
- Use Promise.all for independent parallel operations — don't await sequentially
- async/await is just Promises in disguise — understanding Promises IS understanding async/await
- The Event Loop is the heart of JS — know micro vs macro task ordering
- For React: clean up async operations on unmount using AbortController